

Symbolic Seed Augmentation for Stateful Smart Contract Fuzzing

CSE 5472 Project proposal

Team member. Edison Li

Objective and security impact. I will build a command-line extension that uses symbolic testing to help a smart contract fuzzer explore difficult inputs after reaching meaningful states. I will test whether this improves discovery of violations of developer-specified properties, such as asset conservation or authorization, within a fixed time budget. If successful, the tool will help developers find vulnerabilities before deployment and produce reproducible regression tests.

Related work and limitations. Medusa provides coverage-guided transaction-sequence fuzzing [1], while Halmos supports symbolic testing of Solidity contracts [2]. Input mutation can struggle with restrictive conditions; symbolic exploration can be costly as paths multiply. Optik already augments Echidna's corpus using the Maat symbolic executor [3], and ConFuzzius combines fuzzing, constraint solving, and transaction dependencies [4]. The contribution is a bounded Medusa-Halmos extension and its evaluation; hybrid fuzzing itself is established prior work.

Proposed tool and contribution. The tool will select a concrete transaction prefix from Medusa's corpus and generate a Halmos harness that replays it. Only the next call's arguments become symbolic, with a developer-supplied predicate specifying a desired state or successful operation. I will implement corpus conversion, harness generation, prefix selection, solver-budget control, model decoding, and seed reinsertion. Prefixes will be deduplicated and processed under per-query timeouts and a campaign-wide solver budget. Candidate inputs must reproduce concretely before entering the next fuzzing round. Reaching the target predicate produces an exploration seed; only a separately checked security-property violation counts as a bug.

Scope and completion risks. I will initially support local deployments, fixed actors, scalar ABI types (integers, addresses, and booleans), and prefixes of at most six calls followed by one symbolic call. Risks include execution-environment mismatches, solver timeouts, and unavailable or incompatible benchmarks. I will pin tool versions, preserve deployment, sender, value, and block context, and validate replay and benchmark compatibility early. If integration is slower than expected, I will reduce supported types and targets while retaining the complete solve-replay-reseed workflow. Unsupported cases and timeouts will be reported separately.

Mid-semester update. I will demonstrate the full workflow on two different contracts: extract a prefix, solve a previously unmet target, replay the resulting input, and resume Medusa with the new seed. The update will include the adapter format, initial replay and coverage measurements, and a preliminary baseline comparison. This will demonstrate that the integration works beyond one hand-written example.

Final code artifact and evaluation. I will deliver the CLI, adapters, generated harnesses, replayable tests, and a benchmark runner. I target six to eight contract scenarios, including public bug/fix cases and constructed controls, with provenance recorded and at least one independent contract family reserved from strategy tuning. I will compare unmodified Medusa, Medusa augmented with same-prefix random/boundary inputs, and the symbolic augmentation using identical fixtures, properties, target information, and CPU/time budgets. Five random seeds per configuration will measure reproduced bug discovery, time to first bug, coverage gain, seed replay success, and total overhead including compilation and solving. Runs exhausting the budget without finding bugs will also be reported. Constructed and public cases will be reported separately.

References

1. Trail of Bits. Medusa.
2. a16z crypto. Halmos.
3. Trail of Bits. Optik.
4. Torres et al. ConFuzzius: A Data Dependency-Aware Hybrid Fuzzer for Smart Contracts. IEEE EuroS&P, 2021.